CHAPITRE 2 : INTRODUCTION À JAVASCRIPT

TP : Mes notes du semestre

Objectif

Créer une page web qui permet de saisir ses notes du semestre, de les afficher sous forme de liste colorée, de calculer la **moyenne pondérée** et d'afficher la **mention** correspondante. Ce TP mobilise **toutes les notions du chapitre 2** : variables, conditions, boucles, tableaux, fonctions, objets, manipulation du DOM et événements.

Résultat attendu

La page contient trois zones :

- **un formulaire** : matière, note sur 20, coefficient et bouton « Ajouter » ;
- **une liste des notes** : en vert si la note est ≥ 10, en rouge sinon, avec un bouton ✕ pour supprimer chaque note ;
- **une zone de résultats** : nombre de matières, moyenne et mention.



Figure 1 : la page terminée (la note 23 est refusée : le champ est encadré en rouge)

Fichiers fournis

Récupérez le dossier de départ **tp2-depart.zip**. Le HTML et le CSS sont déjà prêts : **vous écrivez uniquement le JavaScript** dans `js/tp2.js`.

```
tp2/
├── tp2.html        ← fourni (à ne pas modifier)
├── css/
│   └── tp2.css     ← fourni
└── js/
    └── tp2.js      ← votre travail
```

tp2.html (extrait du body) : les identifiants à utiliser

```
<form id="formulaire">
  <label for="matiere">Matière</label>
  <input type="text" id="matiere">
  <label for="note">Note (/20)</label>
  <input type="text" id="note">
  <label for="coef">Coefficient</label>
  <input type="number" id="coef" value="1" min="1">
  <button type="submit">Ajouter</button>
  <p id="erreur"></p>
</form>

<ul id="liste"></ul>

<section id="resultats">
  <p>Nombre de matières : <span id="nombre">0</span></p>
  <p>Moyenne : <span id="moyenne">-</span></p>
  <p>Mention : <span id="mention">-</span></p>
</section>
```

Classes CSS déjà définies dans tp2.css

Classe	Effet
`reussi`	Fond vert clair et bordure verte (note ≥ 10)
`echoue`	Fond rouge clair et bordure rouge (note < 10)
`invalide`	Bordure rouge sur un champ mal rempli
`supprimer`	Style du bouton ✕

Boîte à outils

Ce TP utilise quelques méthodes qui ne sont pas dans le cours. Elles appliquent des notions que vous connaissez déjà : lisez-les avant de commencer et revenez-y si besoin.

Méthode	Rôle	Exemple
`isNaN(valeur)`	Renvoie true si la valeur n'est pas un nombre	`isNaN(Number("abc"))` → true
`nombre.toFixed(2)`	Arrondit à 2 décimales (renvoie du texte)	`(13.6667).toFixed(2)` → "13.67"
`tab.splice(i, 1)`	Retire 1 élément à la position i du tableau	`notes.splice(2, 1)` retire la 3e note
`classList.contains("x")`	Renvoie true si l'élément a la classe x	`li.classList.contains("reussi")`
`formulaire.reset()`	Vide tous les champs du formulaire	`formulaire.reset();`
`e.target.tagName`	Nom de la balise qui a déclenché l'événement, en majuscules	"INPUT", "BUTTON", "LI"…

Méthode	Rôle	Exemple
`data-index="2"`	Attribut personnalisé (data-…) pour ranger une information dans une balise	`btn.setAttribute("data-index", 0)`

> **La délégation d'événements.** Grâce au bouillonnement (vu en cours), un clic sur un bouton remonte jusqu'à ses parents. Au lieu de placer un écouteur sur chaque bouton ×, on en place **un seul sur la liste**, et e.target indique quel bouton a été cliqué. C'est pratique pour des éléments créés en JavaScript, qui n'existent pas encore au chargement de la page.

Pour en savoir plus : cherchez le nom de la méthode sur **developer.mozilla.org/fr**.

Partie A : les bases

Étape 1 Préparer le script

Dans `js/tp2.js`, sélectionnez les éléments de la page dont vous aurez besoin avec `document.querySelector()`, puis créez un tableau vide `notes`. Chaque note sera un **objet** de la forme `{ matiere, note, coef }`.

```
const formulaire = document.querySelector('#formulaire');
// … les autres éléments : #matiere, #note, #coef, #erreur, #liste

const notes = [];   // le tableau qui contiendra toutes les notes
```

Étape 2 Ajouter une note

Écoutez l'événement `submit` du formulaire. Bloquez le rechargement de la page avec `e.preventDefault()`, récupérez les trois valeurs (pensez à `Number()` pour la note et le coefficient), puis ajoutez un objet au tableau avec `push()`.

```
formulaire.addEventListener('submit', function (e) {
  e.preventDefault();
  // récupérer les valeurs…
  notes.push({ matiere: matiere, note: note, coef: coef });
});
```

Testez : ajoutez `console.log(notes)` après le `push` et vérifiez dans la console que le tableau se remplit.

Étape 3 Vérifier la saisie

Avant d'ajouter la note, vérifiez que la matière n'est pas vide et que la note est un nombre compris entre 0 et 20. Si ce n'est pas le cas, affichez un message dans `#erreur` avec `textContent` (pas d'`alert()`) et arrêtez la fonction avec `return`. Si tout est correct, videz le message d'erreur.

Indice : combinez les conditions avec `||` (OU), et utilisez `isNaN()` pour tester si la note est un nombre (voir la boîte à outils).

Étape 4 Afficher la liste des notes

Écrivez une fonction `afficherNotes()` qui :

- vide la liste (`liste.innerHTML = ''`) ;
- parcourt le tableau avec une boucle `for` ;

- crée un `<li>` par note avec `createElement`, le remplit avec un gabarit de chaîne, puis l'ajoute avec `appendChild`.

```
li.textContent = `${n.matiere} : ${n.note}/20 (coef ${n.coef})`;
```

Appelez `afficherNotes()` à la fin de l'étape 2, après chaque ajout.

Étape 5 Colorer les notes

Dans la boucle, ajoutez la classe `reussi` au `<li>` si la note est supérieure ou égale à 10, sinon la classe `echoue`, avec `classList.add()`.

Étape 6 Calculer la moyenne et la mention

Écrivez une fonction `calculerMoyenne()` qui **renvoie** (`return`) la moyenne pondérée :

$$\text{moyenne} = \Sigma\ (\text{note} \times \text{coef}) \ /\ \Sigma\ \text{coef}$$

Écrivez ensuite une fonction `obtenirMention(moyenne)` qui renvoie la mention avec `if / else if / else` :

Moyenne	Mention
16 et plus	Très bien
14 et plus	Bien
12 et plus	Assez bien
10 et plus	Passable
Moins de 10	Ajourné

Affichez le nombre de matières (`notes.length`), la moyenne avec deux décimales (`toFixed(2)`) et la mention dans la zone `#resultats`.

Testez : avec 17 (coef 3), 8.5 (coef 2), 14 (coef 1) et 11 (coef 2), vous devez obtenir **13.00 / 20** et la mention **Assez bien**.

Partie B : aller plus loin

Étape 7 Supprimer une note

Dans `afficherNotes()`, ajoutez dans chaque `<li>` un bouton ✕ avec la classe `supprimer` et un attribut `data-index` qui contient sa position dans le tableau (`setAttribute`).

Placez **un seul écouteur** sur la liste `#liste` : grâce à la **propagation**, un clic sur n'importe quel bouton remonte jusqu'à la liste (*délégation d'événements*, voir la boîte à outils).

```
liste.addEventListener('click', function (e) {
  if (e.target.classList.contains('supprimer')) {
    const index = Number(e.target.getAttribute('data-index'));
    // retirer la note du tableau avec splice(), puis réafficher
  }
});
```

Étape 8 Valider la note en direct

Écoutez l'événement `input` sur le champ note : pendant la saisie, ajoutez la classe `invalide` tant que la valeur n'est pas un nombre entre 0 et 20, et retirez-la sinon (`classList.remove()`).

Étape 9 Ajouter des raccourcis clavier

Écoutez `keydown` sur le `document` et utilisez un `switch (e.key)` :

- `Escape` : vide le formulaire (`formulaire.reset()`) et le message d'erreur ;
- `Delete` : supprime la dernière note ajoutée (`pop()`), **seulement si** l'utilisateur n'est pas en train d'écrire dans un champ (`e.target.tagName !== 'INPUT'`).

Bonus

- Afficher la meilleure et la moins bonne note du semestre.
- Ajouter un bouton « Tout effacer » qui demande une confirmation avec `confirm()` avant de vider le tableau.
- Empêcher d'ajouter deux fois la même matière.

Avant de rendre

- ☐ Chaque étape fonctionne et la console n'affiche aucune erreur.
- ☐ Une note invalide (vide, lettres, 25…) est refusée avec un message clair.
- ☐ La moyenne et la mention se mettent à jour après chaque ajout et chaque suppression.
- ☐ Votre code est indenté et commenté, avec des noms de variables clairs.
- ☐ `const` par défaut, `let` seulement si la valeur change, jamais `var`.

Rendu

Compressez votre dossier `tp2` et déposez-le sur Classroom sous le nom **Nom_Prenom_TP2.zip** avant le **[date limite]**.

> **L'IA est votre assistante, pas votre remplaçante !** Vous devez pouvoir expliquer chaque ligne de votre code.

Annexe : les notions du chapitre dans ce TP

Notion du chapitre 2	Étape(s)
`let` / `const`, types, `Number()`	1, 2
Opérateurs arithmétiques, de comparaison et logiques	3, 6
Chaînes de caractères et gabarits	4
`if / else if / else`	3, 5, 6
`switch`	9
Boucles (`for`, `for…of`)	4, 6

Notion du chapitre 2	Étape(s)
Tableaux (`push`, `splice`, `pop`, `length`)	1, 2, 7, 9
Fonctions (paramètres, `return`)	4, 6
Objets	1, 2
Sélection du DOM (`querySelector`)	1
`textContent` / `innerHTML`	3, 4, 6
Attributs et classes (`setAttribute`, `classList`)	5, 7, 8
Créer et ajouter des éléments (`createElement`, `appendChild`)	4, 7
`addEventListener` : `submit`, `click`, `input`, `keydown`	2, 7, 8, 9
L'objet `event` et `preventDefault()`	2, 7, 9
Propagation des événements (délégation)	7